



RIPHAH INTERNATIONAL UNIVERSITY
Islamabad - I-14 Campus

Faculty of Computing 
Department of Computer Science 



DATABASE MANAGEMENT SYSTEM
Semester Project Proposal

HS BANKING SYSTEM

A Complete Oracle SQL Database Solution



SUBMITTED BY

SYED HASSAN SHAH

SAP ID: 65912

Program: **BSCS (Computer Science)**

Section: **BSCS 4-1** | Semester: **4th**

Instructor: **Mr. Ihtisham Ullah**

Session: **Spring 2026**



PROJECT TITLE

HS BANKING SYSTEM

Implemented Using Oracle SQL

Student Name	Syed Hassan Shah
SAP ID	65912
Program	BS Computer Science (BSCS)
Section	BSCS 4-1
Semester	4th Semester Spring 2026
Course	Database Management System
Instructor	Mr. Ihtisham Ullah
Department	Faculty of Computing
University	Riphah International University, Islamabad I-14

1. Introduction

This project is a semester requirement for the Database Management System course. The goal is to design and build a real, working database system that goes beyond simple queries and shows a proper understanding of database concepts.

The project chosen is the HS Banking System — a complete database for managing all the key operations of a bank. Banks deal with huge amounts of data every day: customer accounts, money transactions, loans, employee records, branch details, and more. Managing all of this data in an organized and reliable way is exactly what a good database system should do.

The system is built using Oracle SQL, following everything practiced in lab sessions. It includes a proper schema design, normalization up to 3NF, an Entity-Relationship Diagram (ERD), all required constraints, and advanced SQL queries such as joins, subqueries, and aggregations. The project is designed to be practical, meaningful, and complex — not just basic CRUD operations.

2. Project Nature

- This is an individual project. All work is done by the student alone.
- The project is unique and has not been copied or shared with any other student.
- It is implemented using Oracle SQL, consistent with the tools used in lab sessions.
- The project covers complex database design, not just simple insert/select queries.
- It is practical and represents a real-world system that is used in the banking industry.

3. Objectives

The main objectives of the HS Banking System project are:

- Design a complete and normalized database schema (up to 3NF) for all banking operations.
- Draw an Entity-Relationship Diagram (ERD) showing all entities, their attributes, and how they are connected.
- Create all database tables in Oracle SQL with proper constraints: PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE, and NOT NULL.
- Use Oracle Sequences to automatically generate unique IDs for accounts, transactions, loans, and cards.
- Write advanced SQL queries including multi-table JOINS, nested subqueries, GROUP BY with HAVING, and date functions.
- Use Oracle Triggers to automate tasks like updating account balances and recording an audit trail of all changes.
- Create Oracle Views to produce useful management reports such as account summaries and branch performance.
- Show meaningful outputs from the database to demonstrate that the system works correctly.

4. Project Description — System Modules

The HS Banking System is divided into eight modules. Each module handles one specific area of the bank's operations and has its own set of database tables connected to the others.

■ Module 1 — Customer Management

This module stores all information about bank customers. Each customer has a unique Customer ID generated automatically by an Oracle Sequence. The data stored includes the customer's CNIC number, full name, date of birth, gender, phone number, email address, home address, and KYC (Know Your Customer) verification status. One customer can have more than one bank account.

■ Module 2 — Account Management

This module manages all bank accounts. There are four types of accounts: Savings, Current, Fixed Deposit, and Business. Every account has a unique account number (created using an Oracle Sequence), an account type, the date it was opened, the current balance, the interest rate, and a status (Active, Frozen, or Closed). A CHECK constraint makes sure the balance can never go below zero. Each account is linked to a customer and a branch.

■ Module 3 — Transaction Processing

Every time money moves in or out of an account, a record is saved in this module. The types of transactions are: Deposit, Withdrawal, Fund Transfer, and Bill Payment. Each transaction stores the transaction ID, date and time, amount, transaction type, the source account, and the destination account (for transfers). An Oracle Trigger runs automatically after each transaction is inserted and updates the account balance in real time.

■ Module 4 — Loan Management

This module tracks the full life cycle of bank loans — from application to full repayment. Loan types include Personal Loan, Home Loan, Car Loan, and Business Loan. For each loan, the system stores the loan ID, customer who applied, employee who processed it, loan type, amount, interest rate, loan period (tenure), approval date, and current status (Pending, Approved, Rejected, or Closed). A separate Loan Repayment table tracks each monthly payment and whether it was paid on time.

■ Module 5 — Branch Management

This module stores information about all bank branches. Each branch has a unique branch code, branch name, city, full address, contact number, and the name of the branch manager. Branch-level reports — such as total deposits, total withdrawals, and loan disbursements — are produced using Oracle Views and GROUP BY queries.

■ Module 6 — Employee Management

This module records information about all bank staff including tellers, loan officers, branch managers, and support staff. For each employee, the system stores an employee ID, name, job title, department, salary, which branch they work at, and their joining date. Reports on how many transactions or loans each employee processed are generated using GROUP BY aggregations.

■ **Module 7 — ATM and Card Management**

This module manages debit and credit cards issued to customers, as well as ATM machine records. Each card has an ID, type (Debit or Credit), spending limit, expiry date, and status (Active, Blocked, or Expired). ATM machines are linked to branches and store the machine's location, available cash, and operational status. A separate ATM Transaction table records all withdrawals made through ATM machines.

■ **Module 8 — Audit and Compliance Logging**

This module automatically keeps a record of every important change made in the database. Oracle Triggers detect any INSERT, UPDATE, or DELETE operations on the Account and Loan tables and save a log entry. Each log entry stores what table was changed, what the action was, who made the change, when it happened, and what the old data looked like. This is important for security, accountability, and meeting banking regulations.

5. Entity-Relationship Diagram (ERD)

An Entity-Relationship Diagram will be drawn to show all the entities in the HS Banking System, their attributes, and the relationships between them. The ERD will be created using standard notation and included in the final printed report.

Main relationships in the ERD:

- A Customer can have many Accounts (one-to-many).
- An Account belongs to one Branch (many-to-one).
- An Account can have many Transactions (one-to-many).
- A Customer can apply for many Loans (one-to-many).
- A Loan has many Repayment records (one-to-many).
- A Branch has many Employees (one-to-many).
- An Employee processes many Loans (one-to-many).
- A Customer can hold many Cards, each linked to one Account (one-to-many).
- A Branch has many ATMs (one-to-many).
- An ATM can have many ATM Transactions (one-to-many).

6. Normalization

Normalization means organizing the database so that data is stored without unnecessary repetition and remains accurate when it is updated. The HS Banking System will be normalized through three stages:

1. **First Normal Form (1NF):** Every column stores only one value (atomic). There are no repeating groups. Each row can be identified by its primary key. For example, a customer's phone numbers are not stored in one column — a separate Contact table is used instead.
2. **Second Normal Form (2NF):** Every non-key column depends on the whole primary key, not just part of it. This applies especially to tables with composite keys. For example, transaction details depend on the full Transaction ID, not just the account number.
3. **Third Normal Form (3NF):** Non-key columns do not depend on other non-key columns. For example, branch city and address depend on BranchID directly — not on the branch manager's name. Lookup values like AccountType and LoanType are stored in separate reference tables.

7. Schema Design — Database Entities

The HS Banking System contains 12 tables. Each table represents one entity. The table below shows every entity, its important columns, and how it connects to other tables.

Entity / Table	Key Columns (PK / FK)	Connected To
Customer	CustomerID (PK-SEQ), CNIC, FullName, DOB, Gender, Phone, Email, KYC_Status	Account, Loan, Card, Notification
Account	AccountNo (PK-SEQ), CustomerID (FK), BranchID (FK), AccountType, Balance, Status, Rate	Transaction, Card, Customer, Branch
Transaction	TxnID (PK-SEQ), AccountNo (FK), BranchID (FK), TxnType, Amount, TxnDateTime	Account, Branch
Loan	LoanID (PK-SEQ), CustomerID (FK), EmpID (FK), LoanType, Amount, Rate, Tenure, Status	Customer, Employee, Loan_Repayment
Loan_Repayment	RepayID (PK-SEQ), LoanID (FK), DueDate, PaidDate, AmountPaid, Status	Loan
Branch	BranchID (PK), BranchCode, BranchName, City, Address, ManagerID (FK)	Account, Employee, ATM, Transaction
Employee	EmpID (PK-SEQ), Name, JobTitle, Department, Salary, BranchID (FK), JoinDate	Branch, Loan
Card	CardID (PK-SEQ), AccountNo (FK), CustomerID (FK), CardType, CreditLimit, Expiry, Status	Account, Customer, ATM_Transaction
ATM	ATMID (PK), BranchID (FK), Location, CashAvailable, Status	Branch, ATM_Transaction
ATM_Transaction	ATM_TxnID (PK-SEQ), ATMID (FK), CardID (FK), Amount, TxnDate	ATM, Card
Audit_Log	LogID (PK-SEQ), TableName, ActionType, ChangedBy, ChangeDate, OldValue	Auto-filled by Triggers
Notification	NotifID (PK-SEQ), CustomerID (FK), Message, SentDate, Channel (SMS/Email)	Customer

8. SQL Queries

The project will include many types of advanced SQL queries to show that the database can answer real banking questions. Simple insert and select queries alone are not enough — the queries below show the full capability of the system.

Query Type	What It Does in the Banking System
Multi-table JOIN (4+ tables)	Show full transaction history with customer name, account type, branch name, and employee — joining 5 tables at once.
Correlated Subquery	Find all customers whose current balance is below the average balance of people with the same account type.
GROUP BY + HAVING	List all branches where the total loan amount given out is more than PKR 10 million this year.
Aggregate Functions	Calculate total deposits, total withdrawals, and average transaction amount for each branch every month.
Oracle Sequence	Automatically create a new unique AccountNo, LoanID, TxnID, or CardID every time a new record is added.
AFTER INSERT Trigger	After every new transaction is added, automatically update the account balance (add for deposit, subtract for withdrawal).
BEFORE DELETE Trigger	Block the deletion of any account that still has a balance greater than zero or has an open loan.
Oracle View	Create a summary view for management that shows active accounts, pending loans, and branch totals — all in one query.
CHECK Constraint	Make sure AccountType is only one of four allowed values, Balance never goes below 0, and Gender is M or F.
Date Functions (SYSDATE etc.)	Find all loan repayments that are overdue — using MONTHS_BETWEEN(SYSDATE, DueDate) to calculate how many months late.
ROLLUP	Produce a pivot-style report of total deposits and withdrawals grouped by Branch, then by AccountType, then overall total.
Audit Trigger (INSERT + UPDATE)	Automatically write a log entry into the Audit_Log table every time data in the Account or Loan table is changed.

■ Sample Query 1 — Top 5 Customers by Total Deposits

```
SELECT    c.CustomerID, c.FullName, SUM(t.Amount) AS TotalDeposits
FROM      Customer c
JOIN      Account a      ON c.CustomerID = a.CustomerID
JOIN      Transaction t ON a.AccountNo  = t.AccountNo
WHERE     t.TxnType = 'Deposit'
GROUP BY  c.CustomerID, c.FullName
ORDER BY  TotalDeposits DESC FETCH FIRST 5 ROWS ONLY;
```

■ Sample Query 2 — Find All Overdue Loan Repayments

```
SELECT    l.LoanID, c.FullName, lr.DueDate,
          ROUND(MONTHS_BETWEEN(SYSDATE, lr.DueDate), 1) AS MonthsOverdue
FROM      Loan l
JOIN      Loan_Repayment lr ON l.LoanID      = lr.LoanID
JOIN      Customer c      ON l.CustomerID    = c.CustomerID
WHERE     lr.Status = 'Pending' AND lr.DueDate < SYSDATE ORDER BY
MonthsOverdue DESC;
```

■ Sample Query 3 — Branch-wise Deposit and Withdrawal Summary (ROLLUP)

```
SELECT    b.BranchName, t.TxnType,
          COUNT(*) AS TotalCount, SUM(t.Amount) AS TotalAmount
FROM      Transaction t
JOIN      Branch b ON t.BranchID = b.BranchID
GROUP BY  ROLLUP(b.BranchName, t.TxnType);
```

9. Outputs

The final submission will include clear, labeled screenshots of the database outputs. Each output will show the result of a specific query or trigger, along with a short explanation of what it shows. The outputs planned include:

- Table structure screenshots (DESC command output for each table).
- Sample data inserted into all 12 tables.
- Output of multi-table JOIN queries showing combined data from 4 or more tables.
- Account balance before and after a transaction (to show the trigger working).
- List of overdue loan repayments with months overdue calculated.
- Branch performance report showing total deposits and withdrawals (ROLLUP output).
- Management dashboard view output showing active accounts, pending loans, and branch totals.
- Audit log entries automatically created by the trigger after account changes.

10. Evaluation Criteria

According to the project instructions, the project will be evaluated on six areas. The table below shows each area and explains how the HS Banking System meets it:

Evaluation Area	How This Project Meets the Requirement
Project Complexity	12 tables, 8 modules, triggers, sequences, views, audit logging — this is an advanced-level project, not a basic one.
Database Design Quality	The schema is normalized to 3NF. All tables have correct primary keys, foreign keys, and CHECK constraints.
Query Implementation	13 types of advanced queries are written: JOINS across 5 tables, correlated subqueries, ROLLUP, triggers, views, and more.
Output Demonstration	All 8 output types listed above are produced with labeled screenshots showing real data from the database.
Viva Performance	The student understands why each design decision was made and can explain the schema, normalization, and every query.
Report Quality	The report includes all required sections: Introduction, Objectives, ERD, Schema, SQL Queries, Outputs, and Conclusion.

12. Conclusion

The HS Banking System is a well-planned, practical, and technically complete database project. It covers all the key areas of a real banking system and uses all the Oracle SQL features required by the course — including normalization, ERD design, constraints, sequences, triggers, views, and advanced queries.

The project is unique, original, and goes well beyond the basic level. It is designed to show that the student understands not just how to write SQL, but how to design a proper database that can handle real-world data in a reliable and organized way.

This project will be taken seriously and completed fully — from the ERD design to the final printed report and viva examination.

Student Signature:

Syed Hassan Shah

SAP ID: 65912 | BSCS 4-1

Instructor Approval:

Mr. Ihtisham Ullah

Faculty of Computing | Spring 2026